Project Report

Scientific Calculator using Java Swing

Submitted by:	[Your name]
Roll number:	[Your roll number]
Course / Branch:	[Course and branch]
College:	[College name]
Guided by:	[Teacher's name]
Date:	[Date]

Contents

1. Abstract
2. Introduction
3. Objectives
4. Tools and Technologies
5. Design of the Calculator
6. How the Program Works
7. Testing
8. Problems I Faced
9. Limitations
10. Future Improvements
11. Conclusion

1. Abstract

This project is a scientific calculator made in Java. It runs as a desktop application and has a dark window with a big display and a grid of buttons, a lot like the calculator that comes with most computers. Along with the normal plus, minus, multiply and divide, it can do powers, roots, logarithms, factorials, trigonometric functions and hyperbolic functions. It also has a button to switch between radians and degrees.

The whole program is written in one Java file and uses only the libraries that already come with Java, so nothing extra has to be installed to run it.

2. Introduction

A calculator looks like a simple thing, but building one is a good way to practise many basic ideas of programming at once: making a window, placing buttons, handling clicks, and turning what the user types into a correct answer.

Normal calculators are good for everyday sums, but students and engineers often need functions such as sin, cos, log and factorial. That is why I decided to build a scientific calculator. I used Java Swing because it is part of Java itself and is a good way to learn how desktop programs are built.

3. Objectives

- To build a working scientific calculator with a clean, easy-to-read screen.
- To make sure the calculator follows the proper order of operations, so 2 + 3 * 4 gives 14 and not 20.
- To support common scientific functions in one place.
- To handle wrong input, such as dividing by zero, without the program crashing.
- To learn how event handling and layouts work in Java Swing.

4. Tools and Technologies

Item	Details
Language	Java (JDK 8 or higher)
GUI toolkit	Java Swing and AWT
Layout	GridBagLayout and BorderLayout
Editor / IDE	[IntelliJ IDEA / Eclipse / VS Code - write the one you used]
Extra libraries	None

5. Design of the Calculator

5.1 Screen layout

The window is split into two parts. The top part is a black display where the numbers and results are shown, right-aligned in a light blue bold font. The bottom part is a grid of buttons with 8 columns and 6 rows.

The buttons are dark grey so they are easy on the eyes. The **C** and **=** buttons are red so they stand out. To get the same shape as a real calculator, the **=** button is two rows tall and the **0** button is three columns wide. Both of these were done using the grid width and height options of GridBagLayout.

5.2 Buttons

Group	Buttons
Digits	0 to 9 and the decimal point
Basic operators	+ - * / = C +/- % ( )
Powers and roots	x^2, x^3, y^x, 2^x, square root, cube root
Other maths	1/x, log, x!
Trigonometry	sin, cos, tan, sinh, cosh, tanh
Extras	pi, Rnd (random number), Rad/Deg switch

6. How the Program Works

6.1 Handling button clicks

The class implements **ActionListener**, so every button sends its click to one method called *actionPerformed*. That method looks at the label of the pressed button and decides what to do. Digits and operators are simply added to a text string that is shown on the display. Buttons like sin or square root work on the number that is currently on the screen and replace it with the answer.

6.2 Calculating the answer

When **=** is pressed, the text on the display (for example *(2+3)*4*) has to be turned into a number. I wrote a small expression evaluator for this. It reads the text from left to right and solves it in layers, so that the right operator goes first:

- Addition and subtraction are handled last.
- Multiplication and division are handled before them.
- Powers are handled before multiplication.
- Numbers and brackets are handled first of all.

Because of this, brackets and normal maths rules work correctly without any extra effort from the user.

6.3 Radians and degrees

The program keeps a true/false variable that remembers whether the calculator is in degree mode. The **Rad** button flips it and changes its own label to **Deg**. Before sin, cos or tan is calculated, the number is converted to radians if degree mode is on.

6.4 Error handling

All the calculation code is placed inside a try-catch block. If something goes wrong, such as dividing by zero, taking the factorial of a negative number, or pressing = on an unfinished expression, the display shows **Error** and the calculator resets instead of closing.

7. Testing

I tested the calculator by entering different inputs and comparing the output with the answer I expected. Some of the tests are shown below.

Input	Expected	Result
2 + 3 * 4 =	14	[Pass / Fail]
(2 + 3) * 4 =	20	[Pass / Fail]
2 y^x 3 =	8	[Pass / Fail]
5, then x!	120	[Pass / Fail]
16, then square root	4	[Pass / Fail]
1000, then log	3	[Pass / Fail]
4, then 1/x	0.25	[Pass / Fail]
Deg mode, 90, then sin	1	[Pass / Fail]
5 / 0 =	Error	[Pass / Fail]

Note: fill in the last column after you run each test on your own computer.

8. Problems I Faced

- **Order of operations.** At first, adding the numbers one after another gave wrong answers for things like 2 + 3 * 4. Writing a proper evaluator that handles each operator level separately fixed this.
- **Button shapes.** Making the = and 0 buttons bigger than the others took a few tries, until I understood how grid width and height work in GridBagLayout.
- **Ugly decimals.** Java prints 8.0 instead of 8, so I added a check that removes the .0 when the answer is a whole number.

- **Special symbols.** Characters like the square root sign did not show properly until the file was saved and compiled as UTF-8.

9. Limitations

- There is no backspace button, so a mistake can only be fixed by pressing C and starting again.
- Answers are stored as double values, so very long decimals can lose a little accuracy.
- There is no history of earlier calculations.
- Keyboard input is not supported; the user has to click the buttons.

10. Future Improvements

- Add a backspace button and keyboard support.
- Add a history panel to see previous calculations.
- Add inverse trigonometric functions ($\sin^{-1}$, $\cos^{-1}$, $\tan^{-1}$), natural log and memory buttons.
- Add a light theme and a way to resize the buttons and text with the window.

11. Conclusion

This project gave me good practice with Java Swing and with turning a real-world tool into a program. The calculator does everything that was planned: it follows the correct order of operations, supports a good range of scientific functions, and does not crash on bad input. Building the expression evaluator was the most useful part, because it showed me how a calculator actually understands what is typed into it.

There is still room to improve it, but as a first version it works well and is easy to use.